

H2. 분포로 보는 시계열

표준편차, 히스토그램, 그리고 z-점수

지난 시간 복습 — H1

- 지난 시간(H1): pandas로 CSV를 불러와 날짜 인덱스를 세우고, 그래프로 3년치 기온을 눈으로 봤다.
- 오늘은 같은 데이터를 분포로 켜다 — 퍼짐을 숫자로 재고, 유난히 튀는 날을 골라내는 법을 배운다.

오늘의 목표

- 표준편차로 데이터의 퍼짐을 잴 수 있다.
- 히스토그램과 상자그림으로 분포의 모양을 읽을 수 있다.
- z-점수로 평균에서 얼마나 떨어졌는지를 재고, 이상치 후보를 찾는 아이디어를 이해한다.
- 튀는 값을 만났을 때 지우기 전에 확인해야 할 것이 있음을 안다.

분포로 본다는 것 — 시간축을 접고 값의 퍼짐 보기

- 지금까지는 가로축이 시간인 그래프를 봤다 — 언제 오르고 언제 내렸는지.
- 오늘은 시간 순서를 잠시 접어두고, 값들이 어떻게 흩어져 있는지를 본다.
- 같은 데이터라도 "언제"를 보는 관점과 "얼마나 퍼져 있는지"를 보는 관점은 서로 다른 것을 알려준다.

평균이 같아도 퍼짐은 다르다

두 공장의 나흘간 하루 생산량이다. (단위: 개)

	1일	2일	3일	4일
공장 A	18	22	18	22
공장 B	14	26	14	26

- 평균을 각각 구해보면 둘 다 **20개**로 같다.
- 평균만 보면 두 공장은 구분되지 않는다 — 그런데 정말 같은 공장일까?

표준편차 — 퍼짐을 재는 자

편차 = 값 - 평균 / 분산 = (편차²)의 평균 / 표준편차 = $\sqrt{\text{분산}}$

- 공장 A: 편차 -2, +2, -2, +2 → 분산 $(4+4+4+4)/4 = 4$ → 표준편차 2
- 공장 B: 편차 -6, +6, -6, +6 → 분산 $(36+36+36+36)/4 = 36$ → 표준편차 6
- 표준편차는 "평균에서 떨어진 정도의 평균적인 크기"다. 공장 B가 훨씬 들쭉날쭉하다.

이제 진짜 데이터로 — 불러오기

```
pd.read_csv(url, parse_dates=["date"], index_col="date")
```

= H1과 같은 세 줄 — url의 CSV를 읽고 date를 날짜 인덱스로 세운다

```
import pandas as pd
```

```
url = "https://raw.githubusercontent.com/jaehoonkim/timeseries-academy/main/data/seoul-temp-daily.csv"  
df = pd.read_csv(url, parse_dates=["date"], index_col="date")  
df.head()
```

describe로 요약하기

```
df["temp_avg"].describe()
```

= temp_avg 열의 요약 통계 여덟 개를 한 번에 — count(개수) / mean(평균) / std(표준편차) / min / 25% / 50%(중앙값) / 75% / max

```
df["temp_avg"].describe()
```

- 3년 1,096일치 데이터를 여덟 개의 숫자로 압축한 것이다.
- std가 바로 앞에서 배운 표준편차다 — 손으로 구할 수도 있지만, describe는 한 줄로 계산해 준다.

히스토그램 — 분포의 얼굴

```
df["temp_avg"].plot(kind="hist", bins=30)
```

= temp_avg를 그리되 / 종류(kind)는 히스토그램 — 값을 구간별로 세어 막대로 / 구간(bins)은 30개

```
df["temp_avg"].plot(kind="hist", bins=30)
```

- 1,096일을 온도 구간(bin)별로 세어 막대로 쌓으면 분포의 모양이 드러난다.
- 막대가 높을수록 그 구간에 머문 날이 많다는 뜻이다.

bins 수와 잔봉우리 — 구간 나누기의 우연

- bins 수를 바꾸면 막대의 폭이 달라지고, 봉우리처럼 보이던 작은 굴곡이 옮겨 다니거나 사라지기도 한다.
- 그러니 작은 잔봉우리를 볼 때는 "진짜 신호인가, 구간을 나누는 우연인가"를 먼저 물어야 한다.
- bins 10과 bins 100으로 같은 데이터를 다시 그려 비교해 보면 어떤 굴곡이 남고 어떤 굴곡이 사라지는지 눈으로 확인할 수 있다.
- bins와 무관하게 끝까지 남는 굴곡이 있다면, 그것이 진짜 분포의 모양이다.

상자그림 — 다섯 숫자 요약

```
df.plot(kind="box")
```

= 각 열을 상자그림으로 — 상자는 값들의 가운데 절반 (25%–75%), 상자 속 가로선은 중앙값, 위아래 수염은 통상 범위, 수염 밖의 동그라미는 이상치 후보

```
df.plot(kind="box")
```

- 상자그림 하나는 최솟값·1사분위수·중앙값·3사분위수·최댓값, 이렇게 다섯 숫자로 분포를 요약한다.
- 수염 밖으로 튀어나온 점(동그라미)이 있다면 이상치 후보다 — 있을 수도, 없을 수도 있다.

z-점수 — $\text{거리} \div \text{보폭} = \text{걸음 수}$

- "평균에서 표준편차 몇 개만큼 떨어졌나"를 재는 값이 z-점수다.
- 두 단계로 구한다: ① 빼기 — 평균에서 떨어진 거리 ② 나누기 — 그 거리를 표준편차라는 자로 다시 재기.
- $\text{거리} \div \text{보폭} = \text{걸음 수}$ 와 같은 구조다. 나누는 순간 단위가 사라지므로, 어떤 데이터끼리도 비교할 수 있는 점수가 된다.
- 표준편차를 자로 쓴 표준 점수 — 기호를 따라 z-점수라 부른다.

z-점수 손 계산 — 공장 B의 어느 날

공장 B(평균 20개, 표준편차 6개)에서 어느 날 생산량이 38개였다고 하자.

1. 빼기 — 평균에서 떨어진 거리: $38 - 20 = +18$ 개

2. 나누기 — 그 거리를 표준편차 6으로 다시 재기: $18 \div 6 = +3$

"평균보다 표준편차 3개만큼 위" — 이 날은 꽤 튀는 날이었다.

z-점수, 1,096일 전체에 — 코드로

```
z = (t - t.mean()) / t.std()
```

= t의 1,096개 값 각각에서 / 평균을 빼고 / 표준편차로 나눈
1,096개의 z-점수 (한 줄이 1,096번의 계산을 대신한다)

```
t = df["temp_avg"]  
z = (t - t.mean()) / t.std()  
z.loc["2024-01-23"]
```

- 방금 손으로 한 계산을, 판다스는 1,096일 전체에 한 줄로 해낸다.
- `z.loc[...]` 로 특정 날짜의 z-점수만 골라볼 수도 있다.

z-점수로 이상치 후보 찾기

```
z[abs(z) > 2]
```

= z 가운데 / 절댓값(abs)이 2보다 큰 값만 골라 달라
(대괄호 안에 조건을 쓰면, 조건이 참인 행만 남는다)

```
z[abs(z) > 2]
```

- 관례상 $|z| > 2$, 즉 "표준편차 2개보다 멀리 떨어진 날"을 이상치 후보로 본다.
- $|z|$ 가 큰 값은 그 자리에서 지울 대상이 아니라, 오타인지 실제 사건인지 확인해야 할 대상이다.
- 며칠이 나오는지, 그 날들에 공통점이 있는지는 활동지에서 직접 확인한다.

오늘 배운 세 문장

1. 표준편차는 퍼짐을 재는 자다.
2. z-점수는 그 자로 잰 거리다.
3. 튀는 값은 지우기 전에 오타인지 사건인지 물어야 한다.

이제 활동지로

1. Colab에서 오늘 배운 코드를 직접 실행해 본다.
2. describe·히스토그램·상자그림으로 분포를 직접 확인한다.
3. z-점수로 이상치 후보를 직접 골라낸다.
4. 오타 실험을 통해 이상치 탐지가 무너지는 경우도 살펴본다.